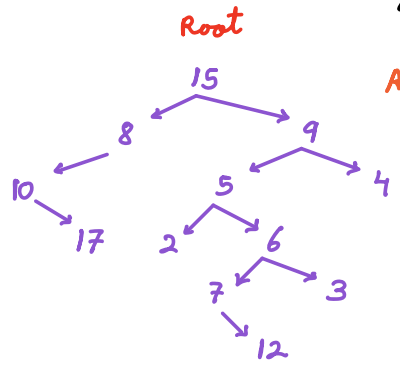
Q → Given the root node of a binary tree.
Count the no. of nodes in the tree.



Ans = 12

$$\boxed{\text{countNode(root)} = \text{countNode(root.left)} + \text{countNode(root.right)} + 1}$$

```
int countNode(Node root){
    if(root == null)
        return 0;
    return ① countNode(root.left) +
           ② countNode(root.right) + 1;   } ③
}
```

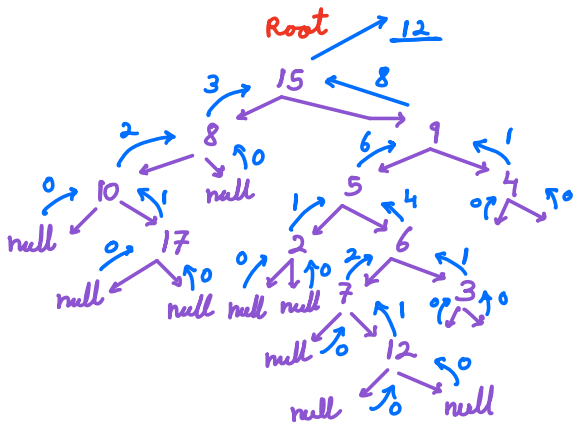
TC = O(N)

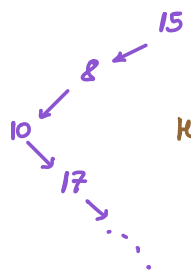
SC = O(1) ✗

O(N) ✗

O(Height) ✓

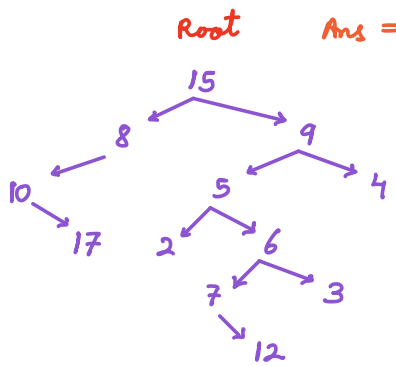
Postorder ✓





Height = N

Q → Find the max data in the given binary tree.

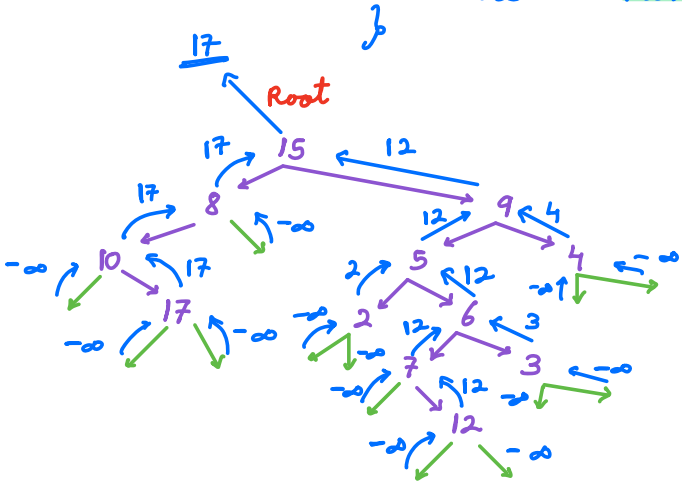


Ans = 17

$$\boxed{\text{maxData(root)} = \max\{\text{maxData(root.left)}, \text{maxData(root.right)}, \text{root.data}\}}$$

```
int maxData (Node root) {
    if (root == null)
        return INT_MIN;
    int maxChildData = max (maxData(root.left),   ①
                            maxData(root.right));  ②
    return max(root.data, maxChildData);
}
```

③



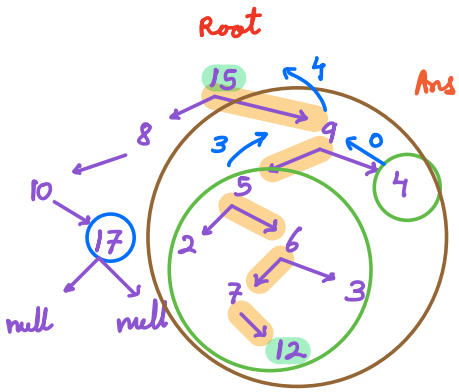
TC = O(N)

SC = O(H)

Postorder

Q → Find the height of the given binary tree.

→ #edges to travel from root to farthest leaf.

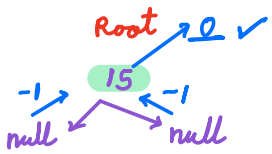


Ans = 5

height (root) = max (height (root.left), height (root.right)) + 1

height(null) = 0 ✗

-1 ✓



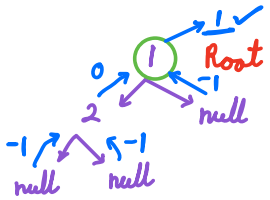

```
int height (Node root) {
    if (root == null)
        return -1;
    return max (height(root.left), height(root.right)) + 1;
}
```

TC = O(N)

SC = O(H)

Postorder

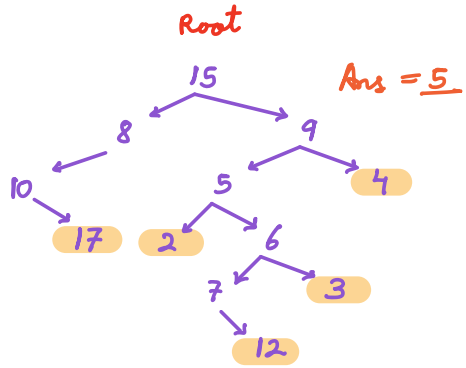
max(a+1, b+1) = max(a, b) + 1



10:20 PM

Q → Count the number of leaf nodes in the tree.

nodes without children.



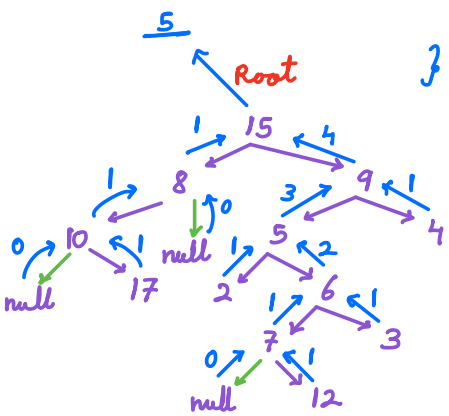
Ans = 5

if (root == null) → return 0
if (root is leaf) → return 1
else → return leaves on left + leaves on right.

TC = O(N)
SC = O(H)
Postorder ✓

```
int countLeaves (Node root) {
    if (root == null)
        return 0;
    if (root.left == null && root.right == null)
        return 1;
    return countLeaves (root.left) + countLeaves (root.right);
}
```

① ② ③

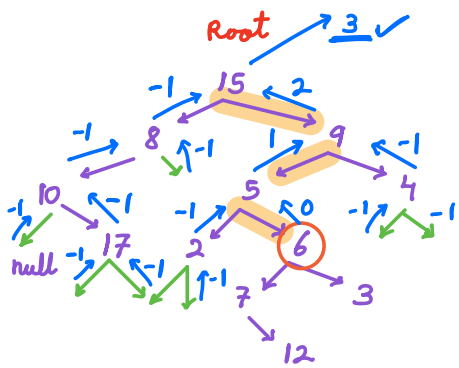


Q → Given a binary tree & a node X, find the depth of node X.
If node X is not present, return -1.
(Nodes are unique)

edges to travel from root to X.

X = 6 Ans = 3

X = 10 Ans = 2



TC = O(N)

SC = O(H)

Postorder

```
int depth(Node root, Node X) {
    if (root == null)
        return -1;
    if (root == X)
        return 0;
    int l = depth(root.left, X);
    int r = depth(root.right, X);
    if (l == -1 && r == -1) {
        return -1;
    }
    return max(l, r) + 1;
}
```

$\underset{③}{\text{max}}(\underset{①}{\text{depth(root.left)}}, \underset{②}{\text{depth(root.right)}}) \underset{④}{==} -1 \; ? \; -1 \; :$

false → $\underset{⑦}{\text{max}}(\underset{⑤}{\text{depth(root.left)}}, \underset{⑥}{\text{depth(root.right)}}) \underset{⑧}{+1}$